



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)



15

Test : CAT- I / CAT- II / Mid-Term

Register No.: 22BCED166 Course Code: BCSE307L Class NBR: 1542
Student's Name: Mr/Ms. CHARAN ADITYA RAVIHANDRAN Course Title: Compiler Design
Programme: B.Tech CSE School: SCOPE Faculty Name: Prof.: SAKAAYA
Slot: C1+TC1 Session: FN/AN/EVN Date of Exam: 27/8/2024 No. of pages written: 16

R.C. Xitir
Signature of the Student

Signature of the Invigilator :

Name / Emp. ID of the Invigilator :

27/8/24
10000

Q No.	Marks	Q No.	Marks	Q No.	Marks	Q No.	Marks	Q No.	Marks
	a b		a b		a b		a b		a b
1	10	9		17		25		33	
2	10	10		18		26		34	
3	5 5	11		19		27		35	
4	9	12		20		28		36	
5	5 5	13		21		29		37	
6		14		22		30		38	
7		15		23		31		39	
8		16		24		32		40	
Sub-Total	39 10	Sub-Total		Sub-Total		Sub-Total		Sub-Total	

Grand Total Marks :

49/50

Evaluator's Signature

Date : 30/8

Evaluator's Name : Prof. Sakaaya A. and Mary S. A

Marks In Words : Four Nine

1. Compiler

It is a program that translates a source input program to an equivalent form in the target language.

Expression $\Rightarrow x = c * d$

Phases of Compiler

The source program, passes through a set of six phases $\Rightarrow$

I) Lexical Analysis

- Breaks down the given program into its constituent tokens and lexemes.

if ($x \geq y$)

{

$x = c * d$;

}

Keyword $\Rightarrow$ if

Operator $\Rightarrow \geq, *, =$

Punctuator $\Rightarrow (,), \{, \}, ;$

Identifier $\Rightarrow x, c, d$

- Suppose we take, $x = c * d$

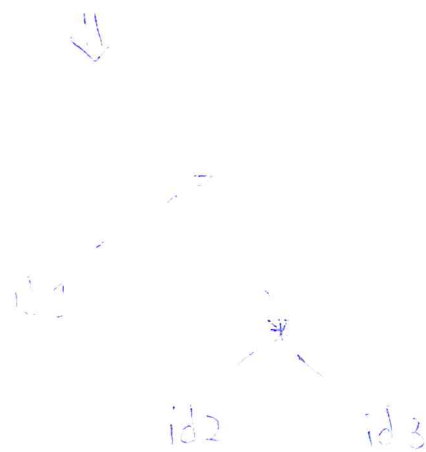
$\Downarrow$

$id1 = id2 * id3$.

I) Syntax Analysis

- Checks for the syntactic correctness of the given expression.
- Generates a parse / syntax tree in the absence of any error

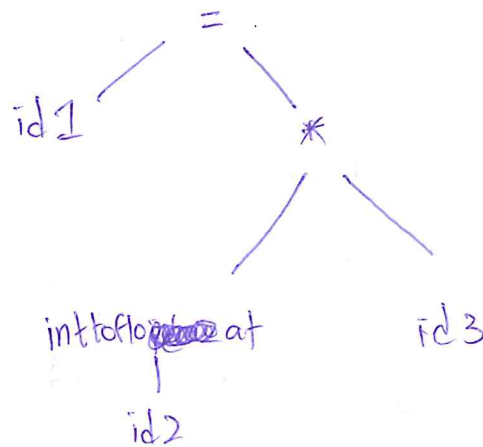
From lexical analysis $\Rightarrow$ $id\ 1 = id\ 2 * id\ 3 ;$



II) Semantic Analysis

- Checks the input from the parser for logical correctness.
- It is responsible for
 - Matching brackets
 - IF-else statements
 - Checking the scope of identifiers

- Cross checking the data types

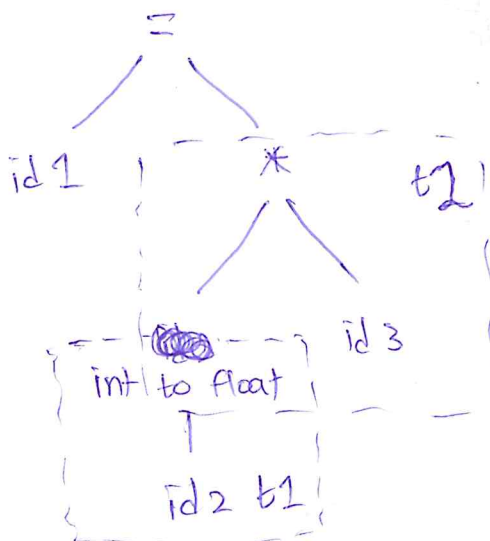


All are real numbers.

IV) Intermediate Code generation

- Main goal $\Rightarrow$ / Properties $\Rightarrow$

- Easy to produce
- Should be easy to translate to target language.



✓) Code Optimization

- Main goal $\Rightarrow$

- Faster execution
- lesser consumption of memory

$t1 = \text{int to float}(id2)$

$t2 = t1 * id3$

$id1 = t2$

$\Rightarrow$

$t1 = (id2)_{\text{float}} * id3$

$id1 = t1$

$\Downarrow$

$id1 = \text{float}(id2) * id3$

✓) Code generation

- Finally, the input program is converted into machine instructions.

MOV $id2, R2$; $id2 \rightarrow R2$

MUL $id3, R2$; $R2 \Rightarrow R2 * id3$

MOV $R2, id1$; $R2 \rightarrow id1$

Thus, the entire program passes through all of the phases of the compiler.

2.

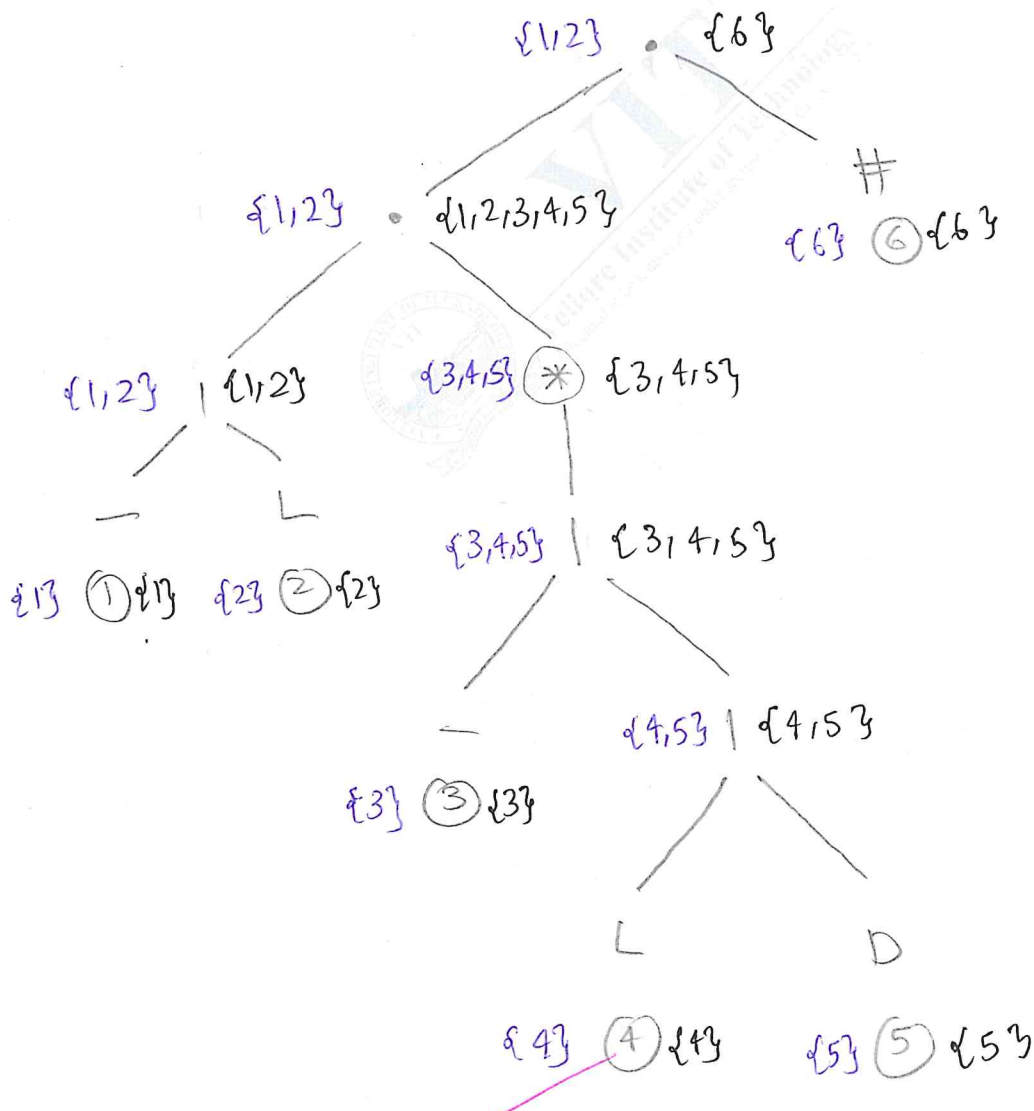
Regular Expression $\Rightarrow (L|D)^*$

$$\therefore RE \Rightarrow (-1L) \cdot (-1L1D)^* \cdot \#$$

Syntax tree $\Rightarrow$

Q3 - firstpos

{ } - Last pos



Followups $\Rightarrow$

<u>Node</u>	<u>Followups</u>
(1) 1	6, 3, 4, 5
(2) 2	6, 3, 4, 5
(3) 3	6, 3, 4, 5
(4) 4	6, 3, 4, 5
(5) 5	6, 3, 4, 5
(6) 6	

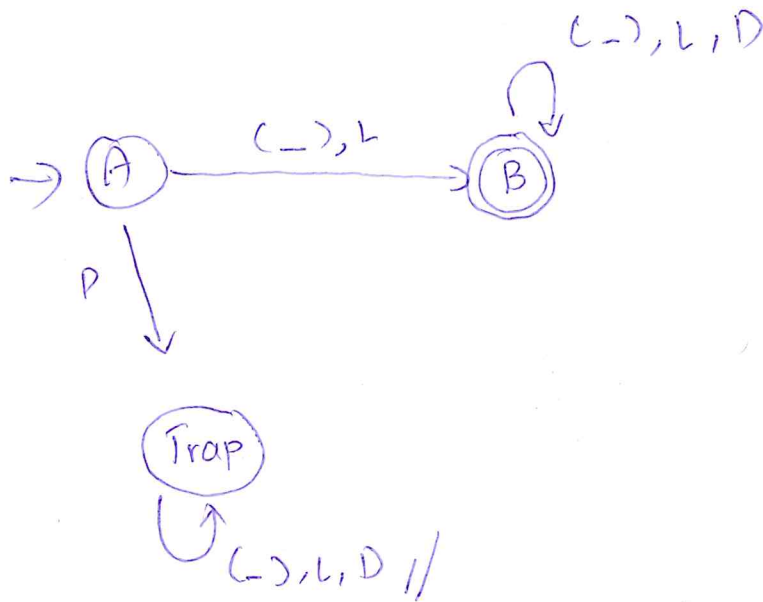
Transitions $\Rightarrow$

Start = FIRST(Root)
node

	—	L	D
A = {1, 2}	{3, 4, 5, 6} B	{3, 4, 5, 6} B	4
B = {3, 4, 5, 6}	{3, 4, 5, 6} B	{3, 4, 5, 6} B	{3, 4, 5, 6} B

2

DFA $\Rightarrow$



10

3.

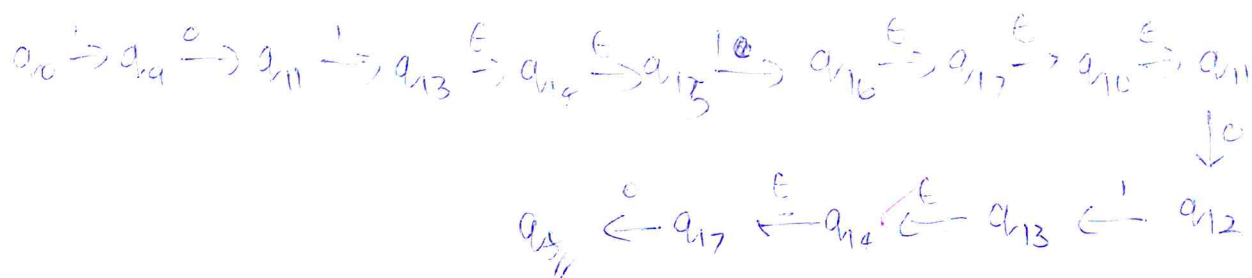
a) $L = \{w \in (0+1)^* \mid w \text{ has no pair of consecutive zeros}\}$

RE $\Rightarrow 0(1^*(10)^*)^* \mid 1((10)^*1^*)^*$
 $0(1^*(10)^*)^* \mid 1((10)^*1^*)^*$

$$RE \Rightarrow 0(1^*(01)^*)^* \mid 1((01)^*1)^*$$



$$w_1 = 1011011$$



Accepted.

$w_2 = 111001$, is not accepted.

b) Regular definition $\Rightarrow$ These are formal notations for patterns for a token.

digit $\rightarrow 0|1|2|3|4|5|6|7|8|9$

digits $\rightarrow \text{digit}(\text{digit})^*$

~~letter~~

letter $\rightarrow a|b|c|\dots|z|A|B|C|\dots|Z$

letters $\rightarrow$ letter (letter)*

identifier $\rightarrow$ (letter | -) (letters | digits | -)* //

Ex : identifier = chararr - 108 //

4. Grammar $\Rightarrow$ $S \rightarrow ACB \mid cbB \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow a \mid \epsilon$

$C \rightarrow h \mid \epsilon$

Since there is no Left Recursion & Non determinism, we can proceed.

Finding FIRST() & FOLLOW()

<u>N.T</u>	<u>FIRST()</u>	<u>FOLLOW()</u>
S	{d, g, h, c, ϵ }	{ $\$$ }
A	{d, g, h, ϵ }	{h, g, $\$$ }
B	{g, ϵ }	{h, g, $\$$, a}
C	{h, ϵ }	{h, g, $\$$ }

LL(1) predictive parsing table $\Rightarrow$

	c	b	a	d	g	h	\$
S	$S \rightarrow cbB$			$S \rightarrow ACB$	$S \rightarrow Ba$ $S \rightarrow ACB$	$S \rightarrow ACB$	$S \rightarrow ACB$
A				$A \rightarrow da$	$A \rightarrow BC$	$A \rightarrow BC$	$A \rightarrow BC$
B			$B \rightarrow \epsilon$		$B \rightarrow \epsilon$ $B \rightarrow \epsilon$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$
C					$C \rightarrow \epsilon$	$C \rightarrow \epsilon$	$C \rightarrow \epsilon$

From the parsing table, it can be observed that the given grammar is not suitable for LL(1) parser construction, since there are positions with **MULTIPLE ENTRIES**.

$\therefore$ NOT SUITABLE.

5. Grammar $\Rightarrow$ $S \rightarrow aABb$,
 a) $A \rightarrow aAc \mid c \mid \epsilon$
 $B \rightarrow d \mid \epsilon$

String $\Rightarrow w = aacddb$

Shift Reduce Parser

- It is a Bottom Up parser
- Tries to obtain the start symbol from the given input string.
- Makes use of two actions $\Rightarrow$
 1. Shifting
 2. Reducing

Parsing through handle pruning $\Rightarrow$

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	aacddb \$	Shift
\$a	acddb \$	Shift
\$aa	ccdb \$	Shift
\$aac	cdb \$	Reduce $A \rightarrow c$
\$aaA	cdb \$	Shift
\$aaAc	db \$	Reduce $A \rightarrow aAc$
\$aA	db \$	Shift

\$ a A d

b \$

Reduce $B \rightarrow d$

\$ a A B

b \$

shift

\$ a A B b

\$

Reduce $S \rightarrow a A B b$

\$ S

\$

ACCEPT



start symbol

Thus the input string $w = aaccd b$ is parsed.

The handles observed $\Rightarrow$

$A \rightarrow c$

$A \rightarrow aAc$

$a \rightarrow d$

$S \rightarrow aABb$

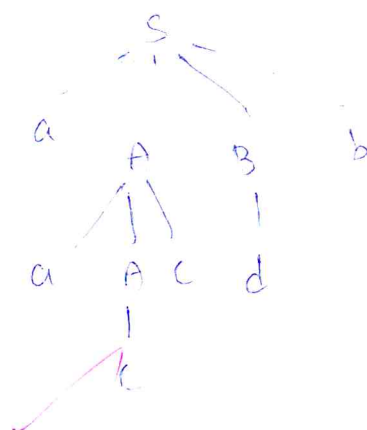
Thus the string $\Rightarrow$

$S \rightarrow aABb$

$A \rightarrow aAc$

$A \rightarrow c$

$B \rightarrow d$



b) Operator Grammar $\Rightarrow$

It has two conditions :

1. There should be no ϵ in the RHS
2. No adjacent Non Terminals on the RHS.

Operator Precedence Parser $\Rightarrow$

It is a parser that uses the bottom up approach.

- Establishes relations between the terminals.
- Tries to generate the start symbol

Two methods :

- 1) Operator Relation table
- 2) Operator Precedence table.

Both of them serve the purpose of establishing a relation between the terminals.

However, the operator table (relation) occupies more space in the memory.

$O(n^2)$ space where $n \rightarrow$ no. of terminals

- Moreover it is a static table based on predefined precedence relations.

On the other hand the Operator Function table $\Rightarrow$

- It assumes the operators involved to be functions.
- Dynamically computes the precedence relations.
- Occupies a much lesser space.

$O(2 \times n)$ where $n \rightarrow$ operators.

• Issue : It establishes relations even in cases that do not exist

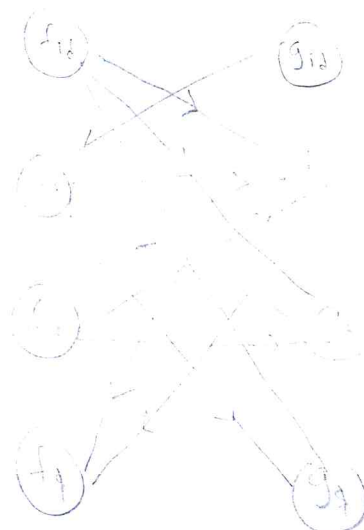
Example $\Rightarrow$

Operator Relation Table $\Rightarrow$

	id	+	*	^
id				
+				
*				
^				

Operator Function Table $\Rightarrow$

	id	+	*	^
id				
+				
*				
^				



all values are filled



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)



21

Test : CAT- I / CAT- II / Mid-Term

Register No.: 22BCE0515 Course Code: BCSE3074 Class NBR: 1542
Student's Name: Mr/Ms. Suyash Verma Course Title: Compiler Design
Programme : B.Tech School : SCOPE Faculty Name : Prof. : Sahaaya Arul Mary S.A.
Slot : C1+TC1 Session : FN/AN/EVN Date of Exam : 27/8/24 No. of pages written : 16

Suyash
Signature of the Student

Signature of the Invigilator : Sahaaya Arul Mary S.A.
Name / Emp. ID of the Invigilator : 100000

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	8		9			17			25			33		
2	5		10			18			26			34		
3	4	4	11			19			27			35		
4	7		12			20			28			36		
5	5	2	13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total	29	6	Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

35/50

Evaluator's Signature

Sahaaya Arul Mary S.A.

Date : 30/8

Evaluator's Name : Prof. Sahaaya Arul Mary S.A.

Marks In Words Three five.

1)

Source program -

```
if (x >= y)
{
    x = c * d;
}
```

The phases in compiler are -

i) Analysis phase :

(a) Lexical Analysis

(b) Syntax Analysis

(c) Semantic Analysis

iii) Synthesis phase

ii) Intermediate code generation

(a) Code optimization

(b) Code generation

Analysis Phase -

(i) Lexical Analysis

if → keyword

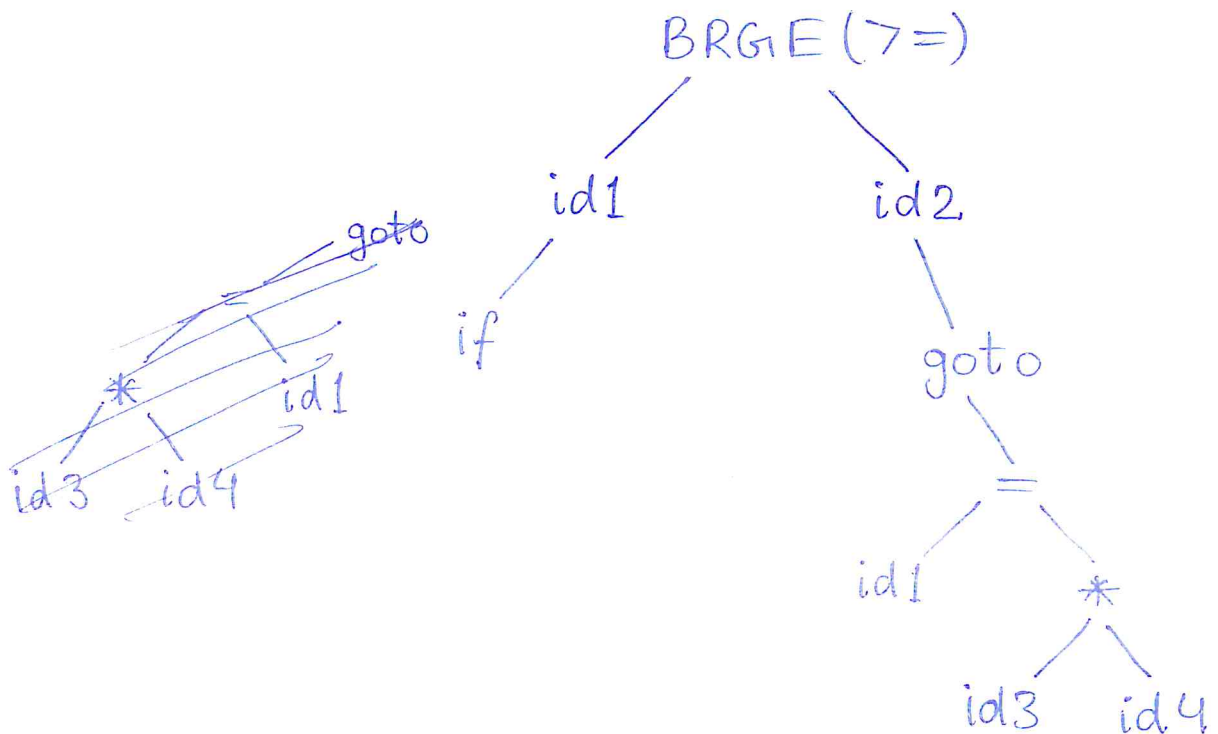
x, y, c, d → identifier

>, =, * → operator

(,), {, } → separator

$\Rightarrow \text{if}(\text{id1} \geq \text{id2}) \text{ goto id1} = \text{id3} * \text{id4}$

(ii) Syntax Analysis



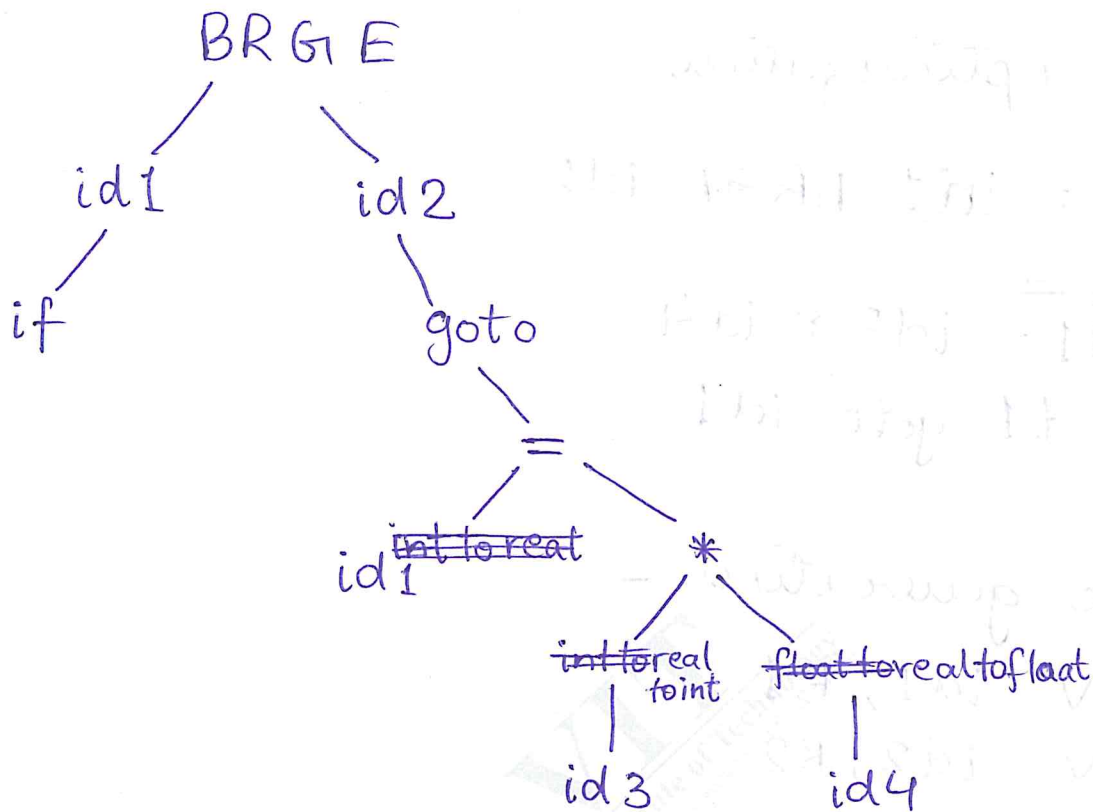
~~iii) Semantic Analysis~~

~~if id1 BRGE id2~~

~~goto id1 = id3 * id4~~

~~Intermediate Code Generation -~~

(iii) Semantic Analysis -



Intermediate Code Generation -

~~if id1 BRGE id2~~

~~goto t1~~

~~t1 =~~

t1 = id1 BRGE id2

t2 = id1

id1 = ~~int to real~~ (id3) * ~~float to real to float~~ (id4)
 real to int real to float

if t1 goto t2

Synthesis phase —

(i) Code optimization

t1 = id1 BRGE id2

~~t2 =~~

id1 = id3 * id4

if t1 goto id1

(ii) Code generation —

MOV id1, R1

MOV id2, R2

~~BRGE R2, R1~~

IF R1 BRGE R2

GOTO TASK

ENDIF

TASK: MOV id3, R3

MOV id4, R4

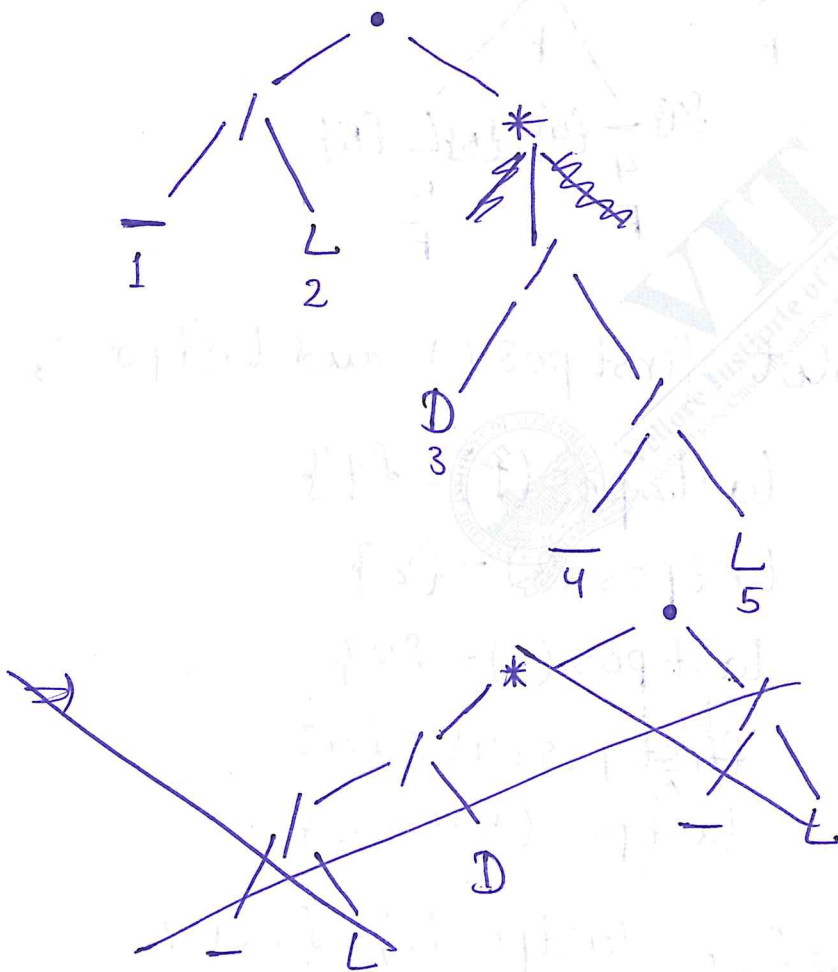
MUL R4, R3

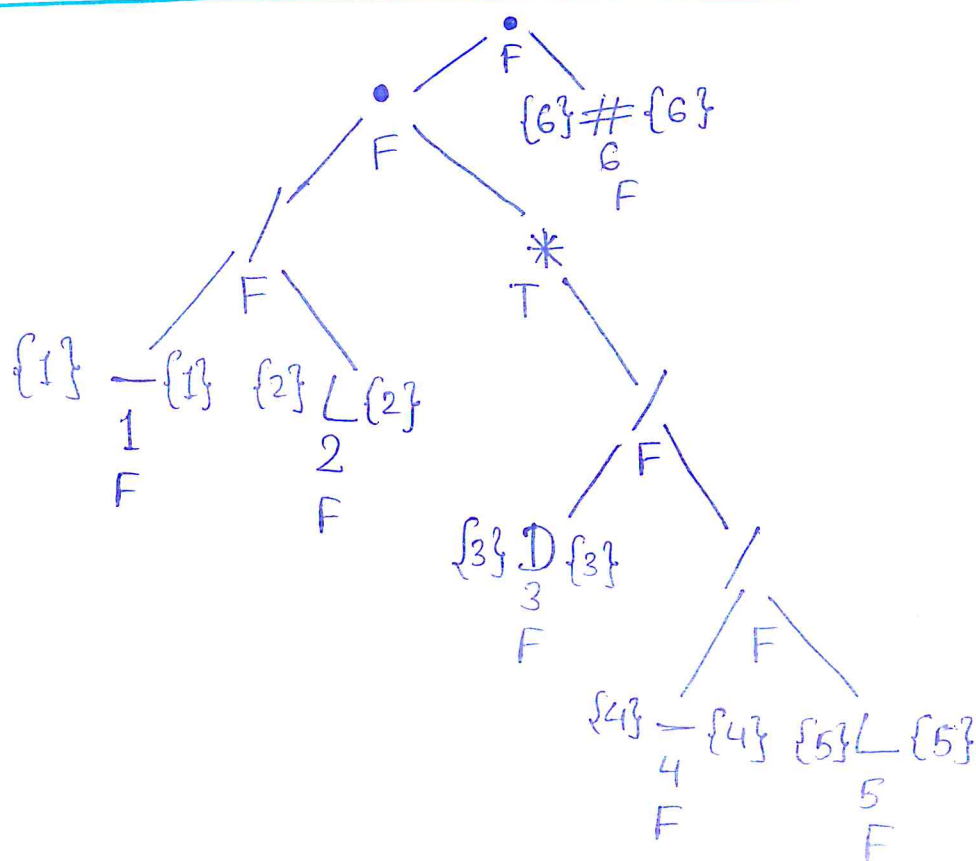
MOV R1, R3

2)

Regular expression = $(-|L)(-|L|D)^*$

Using direct method,





Using rules to plot first pos () and last pos (),

$$\text{firstpos}(\frac{1}{4}) = \{1\} \quad \text{lastpos}(1) = \{1\}$$

$$\text{firstpos}(2) = \{2\} \quad \text{lastpos}(2) = \{2\}$$

$$\text{firstpos}(3) = \{3\} \quad \text{lastpos}(3) = \{3\}$$

$$\text{firstpos}(4) = \{4\} \quad \text{lastpos}(4) = \{4\}$$

$$\text{firstpos}(5) = \{5\} \quad \text{lastpos}(5) = \{5\}$$

$$\begin{aligned} \text{firstpos}(1) &= \{1\} \cup \{2\} & \text{lastpos}(1) &= \{1\} \cup \{2\} \\ &= \{1, 2\} & &= \{1, 2\} \end{aligned}$$

$$\begin{aligned} \text{firstpos}(1) &= \{4\} \cup \{5\} & \text{lastpos}(1) &= \{4\} \cup \{5\} \\ &= \{4, 5\} & &= \{4, 5\} \end{aligned}$$

$$\text{first pos } (/) = \{4, 5\} \cup \{3\} = \{3, 4, 5\} \quad \text{last pos } (/) = \{4, 5\} \cup \{3\} = \{3, 4, 5\}$$

$$\text{first pos } (*) = \{3, 4, 5\} \quad \text{last pos } (*) = \{3, 4, 5\}$$

$$\text{first pos } (.) = \{1, 2\} \quad \text{last pos } (.) = \{3, 4, 5\} \cup \{1, 2\} = \{1, 2, 3, 4, 5\}$$

$$\text{first pos } (.) = \{1, 2\} \quad \text{last pos } (.) = \{6\}$$

Serial Number	followpost()
1	{2, 3, 4, 5, 6}
2	{3, 4, 5, 6}
3	{4, 5, 6}
4	{5, 6}
5	{6}
6	{ }

Let $\{2, 3, 4, 5, 6\} \rightarrow A$, $\{3, 4, 5, 6\} \rightarrow B$, $\{4, 5, 6\} \rightarrow C$,
 $\{5, 6\} \rightarrow D'$, $\{6\} \rightarrow E$

~~$\Rightarrow (A, -) =$~~

$$(A, -) = \{5, 6\} = D' \quad (B, -) = D'$$

$$(A, L) = \{3, 4, 5, 6\} = B \quad (B, L) = E$$

$$(A, D) = \{4, 5, 6\} = C \quad (B, D) = C$$

$$(C, -) = D' \quad (D', -) = \phi$$

$$(C, L) = E \quad (D', L) = E$$

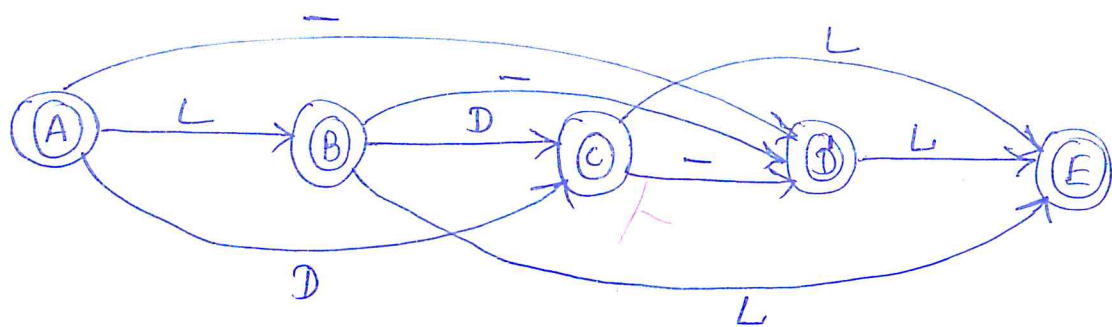
$$(C, D) = \phi \quad (D', D) = \phi$$

$$(E, -) = \phi$$

$$(E, L) = \phi$$

$$(E, D) = \phi$$

Thus, DFA is



This is the DFA of regular expression.

$$4) S \rightarrow ACB | cbB | Ba \text{ } \$ \text{ } \cancel{ACB}$$

$$A \rightarrow da | BC$$

$$B \rightarrow g | \epsilon$$

$$C \rightarrow h | \epsilon$$

$$\begin{array}{l} S \rightarrow ACB' | cbS' | s'a \\ S' \rightarrow B \end{array}$$

$$\text{FIRST}(\overset{C}{\$}) = \{h, \epsilon\}$$

$$\text{FIRST}(B) = \{g, \epsilon\}$$

$$\text{FIRST}(A) = \{d, g, \epsilon, h\}$$

$$\text{FIRST}(S) = \{d, g, c, \epsilon, h, a\}$$

$$\text{FOLLOW}(S) = \{\emptyset, \$\}$$

$$\text{FOLLOW}(B) = \{\$, a, h\}$$

$$\text{FOLLOW}(\overset{C}{\$}) = \{g, h\}$$

$$\text{FOLLOW}(A) = \{h, g\}$$

Thus

	First	Follow
S	{d, g, a, c, h, ϵ }	{ $\emptyset$, \$}
A	{d, g, h, ϵ }	{$\emptyset$, a, h} {h, g}
B	{g, ϵ }	{ $\emptyset$, a, h}
C	{h, ϵ }	{g, h}

?

~~For S →~~

Inputs

a b c d g ~~h~~ h \$

~~S → cb~~ S → ACB S → ACB S → ACB

S

A

B

C

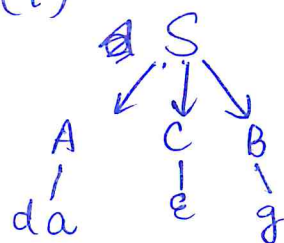
For S → ACB

FIRST(ACB) = {d, g, ε, h}

	a	b	c	g	d	h	h \$
S			S → cbB	S → ACB S → Ba	S → ACB	S → ACB	
A				A → BC	A → da		
B	B → ε			B → g		B → ε	B → ε
C				C → ε		C → h C → ε	

Thus, as there are two rules in single space in $M(A, S, g)$ and $M(C, h)$. This grammar is not suitable for LL(1) construction.

Thus, string $w = dag$ is accepted by grammar.



5)(a) $S \rightarrow aABb$

$A \rightarrow aAc \mid c \mid \epsilon$

$B \rightarrow d \mid \epsilon$

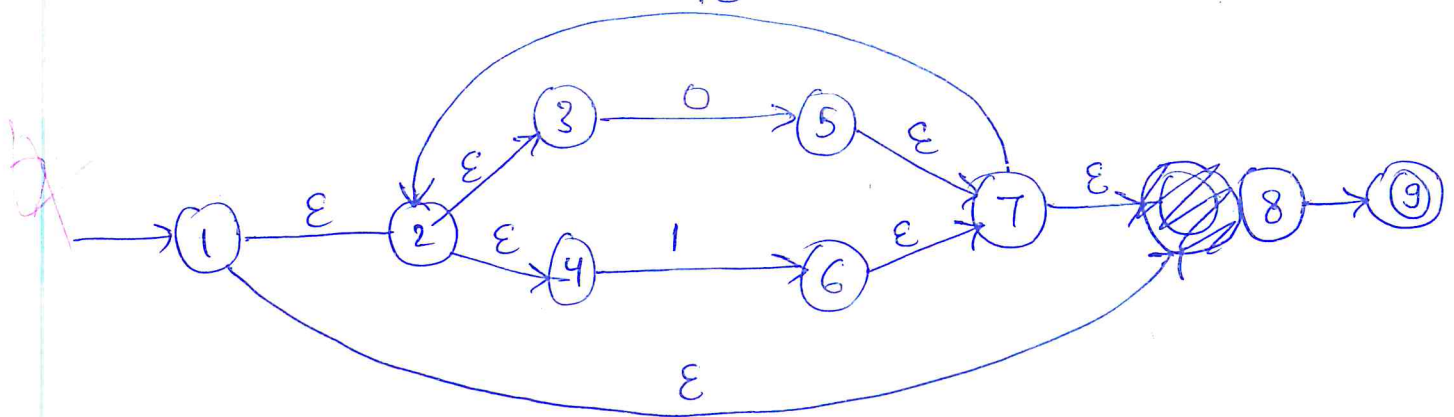
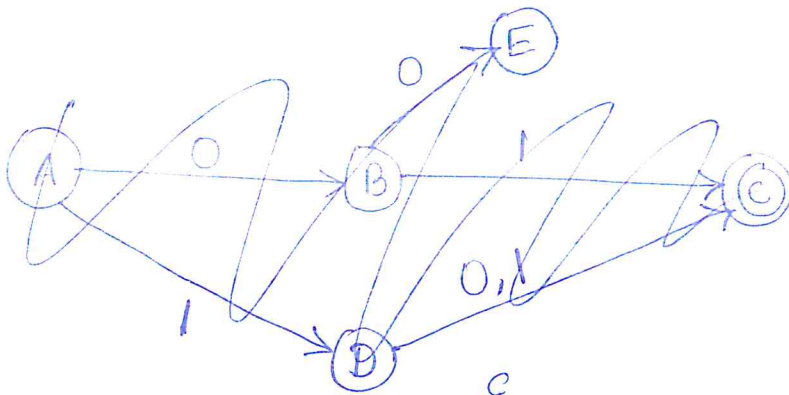
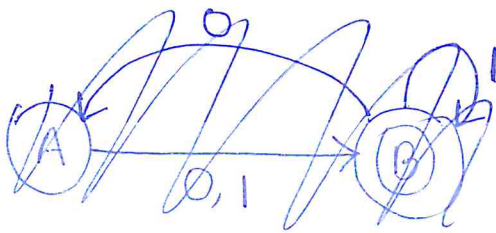
Stack	Input	Action
\$	aacdb\$	shift
\$a	a acdb\$	shift
\$aa	a ccdb\$	shift
\$aaa	cedb\$	shift
\$ a aac	cdb\$	Reduce, $A \rightarrow c$
\$ a aaA	cdb\$	Shift
\$ a aaAc	db\$	Reduce, $A \rightarrow aAc$
\$ a aA	db\$	Shift
\$ a aAd	b\$	Reduce, $B \rightarrow d$
\$ a aAB	b\$	Shift
\$aABb	\$	Reduce, $S \rightarrow aABb$
\$S	\$	Accept

Thus, string is accepted.

3) (a) $L = \{ w \in (0+1)^* \mid w \text{ has no cons. zeros} \}$

~~Ans~~

Let finite automata be



w_1 is accepted by L .

w_2 is not accepted by L as it has ~~two~~ a pair of continuous zeroes in it.

3) (b)

Regular definition :

for letters $((a-z)^* | (A-Z)^*)^*$

for digits $(0-9)^*$

only underscore $(-)^*$

⇒ Regular definition —

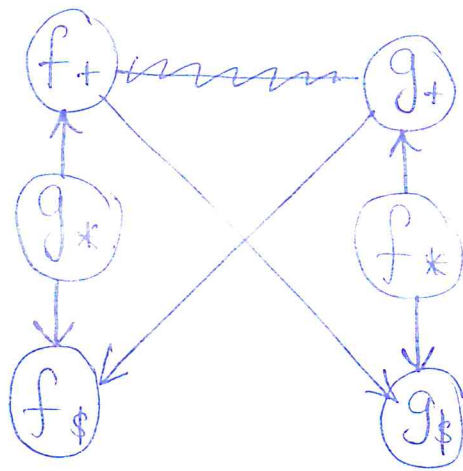
4 ~~$((a-z) | (A-Z))^* (0-9)^*$~~

$(- | (a-z) | (A-Z)) ((a-z) | (A-Z))^* (0-9)^* (-)^*$

5) (b) Let operator relation table be,

	+	*	\$
+		<	>
*	>		>
\$	<	<	

for two function f & g ,



	+	*	\$
f	1	2	0
g	1	2	0

Using this example we can see that in operator relation table we can directly see the relation between various operators whereas in operator function table we see numbers assigned to each operator. These numbers show the precedence order of operators, the higher the number the higher the precedence. Here '*' has highest precedence and '\$' has lowest precedence.

Seat No.



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Authorization
Seal

Test : CAT- I / CAT- II / Mid-Term

Register No.: 22 BCT 0133 Course Code: BCSE307L Class NBR:
Student's Name: Mr/Ms. Chirag Gunti Course Title: Compiler Design
Programme : B.TECH School : SCOPE Faculty Name : Prof. :
Slot : C1 Session : EN/AN/EVN Date of Exam : 27/8/24 No. of pages written :

Chirag

Signature of the Student

Signature of the Invigilator : [Signature]

Name / Emp. ID of the Invigilator : 14492

Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks		Q No.	Marks	
	a	b		a	b		a	b		a	b		a	b
1	0		9			17			25			33		
2			10			18			26			34		
3		1	11			19			27			35		
4	4	1	12			20			28			36		
5	5	1	13			21			29			37		
6			14			22			30			38		
7			15			23			31			39		
8			16			24			32			40		
Sub-Total	9	2	Sub-Total			Sub-Total			Sub-Total			Sub-Total		

Grand Total Marks :

11/50

Marks In Words : One One

Evaluator's Signature

Date : 30/8

Evaluator's Name : Prof. Sahaya Anul Mary

Q4) given grammar (G):

$$\begin{array}{l} S \rightarrow ACB | cbB | Ba \\ A \rightarrow da | BC \\ B \rightarrow g | \epsilon \\ c \rightarrow h | \epsilon \end{array}$$

L4(1) condition states that the given grammar should not have left recursion or left factoring at the production to be suitable for parsing. We shall check this now.

Left recursion: None of the non-terminals are occurring leftmost of the production where the start symbol is ~~same~~ same for Ex: $S \rightarrow SaB$ X
Our grammar is not having this issue.

Left factoring: none of the productions have more than one leftmost terminal that is the same EX: $A \rightarrow aB | aC$ X

Our grammar is free from this issue.

Thus, Our grammar is fit for predictive parsing techniques.

To construct predictive parsing table, we need to calculate $\text{first}()$ and follow tuples for non-terminal productions.

$$S \rightarrow ACB | cbB | Ba$$

$$A \rightarrow da | BC$$

$$B \rightarrow g | \epsilon$$

$$C \rightarrow h | \epsilon$$

$$\text{First}(S) = \{a, c, b, d, g, \epsilon\}$$

$$\text{First}(A) = \{d, a, g, \epsilon, h\}$$

$$\text{First}(B) = \{g, \epsilon\}$$

$$\text{First}(C) = \{h, \epsilon\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{h, g, \$ \}$$

$$\text{Follow}(B) = \{a, \$, h, g\}$$

$$\text{Follow}(C) = \{g, \$, h\}$$

	First	Follow
S	{a, b, c, d, g, ϵ }	{ ϵ }
A	{a, d, g, h, ϵ }	{g, h, \$}
B	{g, ϵ }	{a, g, h, \$}
C	{h, ϵ }	{g, h, \$}

productive parsing table

	a	b	c	d	g	h	\$
S		$S \rightarrow B$	$S \rightarrow B$				$S \rightarrow E$
A							
B					$B \rightarrow g$	$B \rightarrow E$	$B \rightarrow E$
c				$c \rightarrow E$	$c \rightarrow h$	$c \rightarrow E$	
D							

Q5) a) $S \rightarrow aABb$
 $A \rightarrow aAc|c|\epsilon$
 $B \rightarrow d|\epsilon$

Buffer	Input	Action
\$	aacdb\$	shift
\$a	aecdb\$	shift
\$aa	ecdb\$	shift
\$aac	cdb\$	Reduce
\$aaA	cdb\$	shift
\$aaAc	db\$	Reduce
\$aA	db\$	shift
\$aAd	b\$	Reduce
\$aAB	b\$	shift
\$aABb	\$	Reduce
\$S	\$	<u>Accept</u>

all possible
handles.

PTO $\Rightarrow$

$$b) S \rightarrow A + B \mid A$$

$$A \rightarrow B * C \mid B$$

$$B \rightarrow id$$

	leading	trailing
S	{+, *, id}	{+, *, id}
A	{*, id}	{*, id}
B	{id}	{id}

Op - relation over relation rules.

op- function that maps relation to Non-terminals.

op-relation	+	*	id
+		<	<
*			<
id			

Q3) b)

identifizier : $(\text{und} / \text{let})^+ (\text{dig} / \text{let} / \text{und})^*$

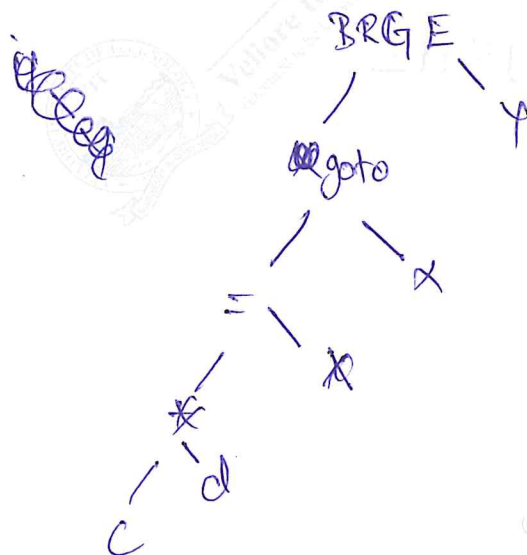
und $\Rightarrow$ $| _ |$

let $\Rightarrow$ $| a-z / A-Z \dots |$

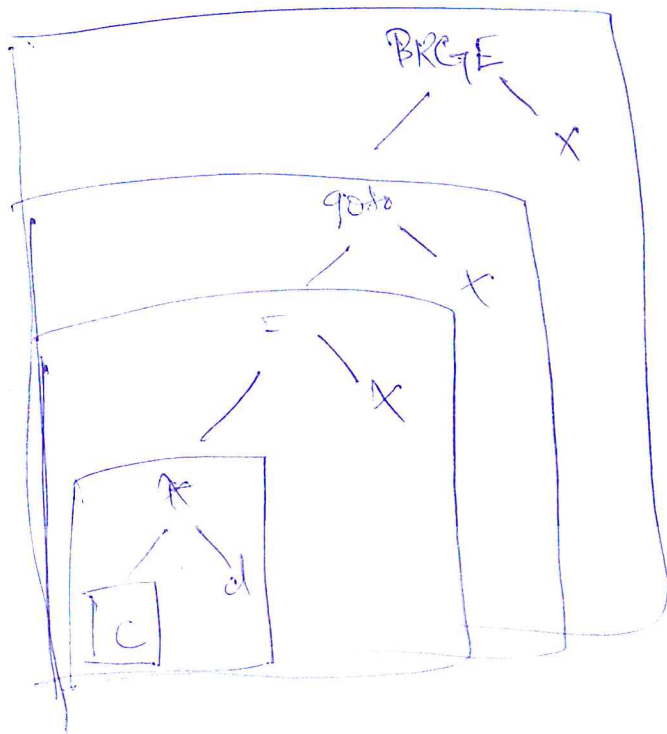
dig $\Rightarrow$ $| 0-9 \dots |$

Q1)

syntax analysis :



intermediate code generation



$c = id1$

$int1 = c * d * id1$

$int2 = X$

~~$int2$~~ $X = id1 * id2$

$id2 = d$

int

✓